

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆที่เกี่ยวข้องในการวิจัย และพื้นฐานของระบบ อิมเมจเสิร์จเอนจิน ซึ่งเนื้อหาในบทนี้จะกล่าวถึงเรื่องของเสิร์จเอนจิน และเรื่องของข้อมูลภาพ ดิจิตอล ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษาระบบอิมเมจเสิร์จเอนจิน

2.1 กลไกการทำงานของเสิร์จเอนจิน

เสิร์จเอนจินทำงานโดยการเก็บข้อมูลเกี่ยวกับหน้าเว็บเพจ ซึ่งเสิร์จเอนจินสามารถดึงเอา ข้อมูลออกมาจากตัวเว็บไซต์ได้เอง โดย Web crawler จะทำหน้าที่ในการดึงข้อมูลของหน้านั้นมา เก็บไว้ ข้อมูลของเว็บแต่ละหน้าจะถูกวิเคราะห์เพื่อพิจารณาว่าจะเก็บเป็นอินเด็กซ์อย่างไร ข้อมูล เกี่ยวกับเว็บเพจจะถูกเก็บไว้ในฐานข้อมูลอินเด็กซ์เพื่อค้นหาในภายหลัง สำหรับเสิร์จเอนจินบาง ตัวอย่างเช่น Google จะเก็บทุกๆส่วนของ Source Page

เมื่อผู้ใช้งานใช้งานเสิร์จเอนจิน และทำการถามหรือค้นหา ตัวอย่างเช่น การใส่ keyword เพื่อค้นหาซึ่งเสิร์จเอนจินจะทำการมองหาอินเด็กซ์ เพื่อจัดเตรียมรายชื่อของหน้าเว็บที่ ตรงกับความต้องการมากที่สุดโดยใช้กฎเกณฑ์ของเสิร์จเอนจิน แต่ก็มีเสิร์จเอนจินอีกแบบที่ไม่ได้ ใช้การเก็บอินเด็กซ์ อย่างเช่น Orase ข้อมูลที่เสิร์จเอนจินต้องการนั้นใช้เมื่อเกิดการถามขึ้นเท่านั้น ซึ่งจะมีข้อดีตรงที่ข้อมูลจะมีการอัปเดตมากกว่า ทำให้ไม่มี dead link แต่มีข้อเสียคือใช้เวลาในการ ค้นหาช้ากว่า

โดยสรุปแล้วอาจแบ่งกระบวนการทำงานของเสิร์จเอนจินออกเป็น 3 ส่วน คือ

- 1) โปรแกรมรวบรวมเอกสารเว็บ (Spider หรือ Crawler) จะทำหน้าที่ออกไปสำรวจเว็บ จากโดเมนต่าง ๆ โดย Spider จะเข้าไปสำรวจและอ่านเว็บเพจ และหากพบลิงก์ก็จะทำ การติดตามลิงก์ภายในเว็บไซต์จนครบ ซึ่งจากการทำงานในลักษณะโง่งมนี้ จึงเป็น ที่มาของคำว่า Spider หรือ Crawler จากนั้น Spider จะนำข้อมูลเว็บดังกล่าวไปเก็บไว้ใน ฐานข้อมูลของเสิร์จเอนจิน และ Spider จะกลับไปตรวจสอบข้อมูลในเว็บนั้นๆอย่าง สม่าเสมอ เช่น ทุก 1 หรือ 2 เดือนเพื่อสำรวจความเปลี่ยนแปลง
- 2) รายการบรรณานุกรมข้อมูลที่ Spider พบจะถูกส่งต่อมาจัดเก็บที่รายการบรรณานุกรม (อินเด็กซ์ หรือ catalog) ซึ่งเปรียบเสมือนเป็นสมุดเล่มใหญ่ที่จัดเก็บสำเนาของเว็บเพจที่ Spider พบ หากข้อมูลที่เว็บต้นฉบับมีการเปลี่ยนแปลงข้อมูลในสมุดบรรณานุกรมจะเปลี่ยนแปลง ด้วย เอกสารเว็บจะถูกทำบรรณานุกรมและจัดเก็บตามบัญชีบรรณานุกรมที่กำหนดไว้

3) โปรแกรมการสืบค้น (เสิร์จเอนจิน ซอฟต์แวร์) จะทำหน้าที่ในการค้นหาข้อมูลจากฐานข้อมูลของเสิร์จเอนจิน จะเริ่มต้นการทำงานเมื่อผู้ใช้ป้อนคำค้นหาโปรแกรมจะทำหน้าที่นำคำค้นหาของผู้ใช้ไปจับคู่กับดัชนีในฐานข้อมูลแล้วทำการดึงข้อมูล (เอกสารเว็บ) ที่ตรงกับคำค้นหาออกมา และจัดลำดับผลการค้นหาตามระดับความเกี่ยวข้องที่โปรแกรมประเมินได้ซึ่งเสิร์จเอนจินแต่ละตัวจะใช้วิธีการที่แตกต่างกันไป ตัวอย่างการสร้างเกณฑ์ในการกำหนดค่าน้ำหนักให้กับคำดัชนีของเสิร์จเอนจินอาจจัดลำดับ ดังนี้ (ลำดับที่ 1 จะให้ค่าน้ำหนักความเกี่ยวข้องกับคำค้นหามากที่สุด)

- ลำดับที่ 1 คำหรือวลี ที่ปรากฏใน Meta tag elements
- ลำดับที่ 2 คำหรือวลี ที่ปรากฏใน Title tag
- ลำดับที่ 3 คำหรือวลี ที่ปรากฏใน Main heading และ Sub heading
- ลำดับที่ 4 จำนวนครั้งที่คำ หรือวลี ปรากฏในส่วนเนื้อหาของเอกสาร
- ลำดับที่ 5 ความถี่ที่เว็บอื่น ๆ เชื่อมโยงเข้ามา

2.2 ประเภทของบริการค้นหาเว็บ

บริการค้นหาเว็บเพจที่มีอยู่ในปัจจุบันเช่น Google, AltaVista, Lycos, Yahoo, HotBot มักเรียกโดยรวมว่าเป็นเสิร์จเอนจินนั้นแท้ที่จริงแล้วสามารถแบ่งประเภทของเครื่องมือที่ช่วยในการค้นหาเว็บเพจได้เป็น 3 ประเภท ดังนี้

2.2.1 เสิร์จเอนจิน

เป็นระบบซอฟต์แวร์ที่มีโปรแกรมโรบอท (Robot) ซึ่งบางครั้งเรียกว่า สไปเดอร์ (Spider) หรือ ครอว์เลอร์ (Crawler) ทำหน้าที่เดินทางไปยังเว็บไซต์ต่างๆ ในอินเทอร์เน็ต และอ่านเว็บเพจจากไซต์เหล่านั้นเพื่อนำมาสร้างดัชนีรายการของเว็บเพจโดยอัตโนมัติ การทำดัชนีรายการเว็บเพจด้วยเสิร์จเอนจินจึงสามารถสร้างดัชนีของเว็บเพจได้เป็นจำนวนที่มากกว่าของไครเรทอรี เนื่องจากดัชนีของไครเรทอรีจะถูกจัดการโดยมนุษย์ แทนที่จะใช้คอมพิวเตอร์ ถ้าหากเว็บเพจที่ถูกทำดัชนีแล้วเกิดมีการเปลี่ยนแปลงเสิร์จเอนจินจะทราบถึงการเปลี่ยนแปลงและจัดนำเว็บเพจที่มีการเปลี่ยนแปลงนั้นมาสร้างดัชนีใหม่ ซึ่งการสร้างดัชนีใหม่นี้อาจส่งผลต่อการจัดลำดับของเว็บเพจนั้น ตัวอย่างของเสิร์จเอนจินที่มีอยู่ในปัจจุบันเช่น HotBot, AltaVista เป็นต้น

2.2.2 ไดรเรทอรี (Directory)

แตกต่างจากเสิร์จเอนจินตรงที่ไดเรทอรีถูกจัดการโดยมนุษย์ เว็บไซต์ใดที่ต้องการมีรายชื่อในไดเรทอรีต้องติดต่อผู้ดูแลไดเรทอรีเพื่อให้ผู้ดูแลไดเรทอรีจำแนกประเภทของเว็บเพจนั้นๆให้อยู่ในประเภทที่เหมาะสม เนื่องจากไดเรทอรีถูกจัดเป็นหมวดหมู่ การค้นหาเว็บเพจด้วยไดเรทอรีจึงให้ผลลัพธ์ที่ตรงกับความต้องการมากกว่าเสิร์จเอนจิน ถ้าหากมีการเปลี่ยนแปลงเกิดขึ้นกับเว็บเพจที่ได้รับการจัดลำดับแล้วการเปลี่ยนแปลงนั้นจะไม่ส่งผลกระทบต่อลำดับเดิมที่ได้รับการจัดไว้ ตัวอย่างไดเรทอรีที่มีอยู่ในปัจจุบันเช่น Yahoo

2.2.3 ไฮบริดเสิร์จเอนจิน (Hybrid Search Engine)

เป็นเสิร์จเอนจินที่มีไดเรทอรีผสมอยู่ด้วย

2.3 องค์ประกอบของเสิร์ชเอนจิน

เสิร์ชเอนจินประกอบด้วย 3 ส่วนหลัก ได้แก่ ไรบอท อินเด็กเซอร์และเสิร์ชเอนจินซอฟต์แวร์

2.3.1 ไรบอท (robots) หรือ สไปเดอร์ หรือ ครอว์เลอร์

ทำหน้าที่เดินทางไปยังเว็บไซต์ต่างๆเพื่อสะสมไฟล์ HTML เพื่อมาเป็นข้อมูลสำหรับสร้างดัชนีค้นหา โดยทั่วไปแล้วไรบอทจะรวบรวมเอกสารเฉพาะเว็บไซต์ที่ถูกกำหนดอยู่ก่อนแล้ว หลังจากที่ได้อ่านเว็บเพจเรียบร้อยแล้วไรบอทจะกลับมาอ่านเว็บไซต์ที่อ่านไปแล้วเพื่อตรวจสอบการเปลี่ยนแปลงตามระยะเวลาที่กำหนด ซึ่งโดยทั่วไปแล้วจะประมาณ 2 เดือน

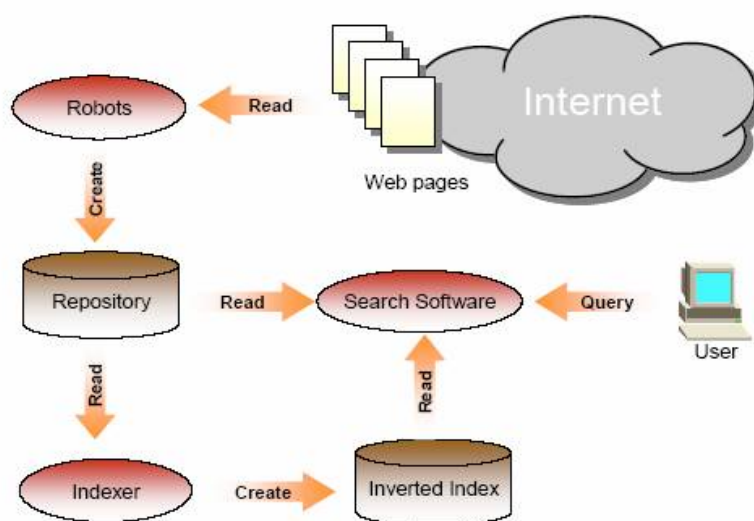
2.3.2 อินเด็กเซอร์ (Indexer, Catalog)

เป็นส่วนที่ทำหน้าที่สร้างดัชนีค้นหาจากไฟล์ HTML ที่ไรบอทหามา วิธีการสร้างดัชนีค้นหานี้มีอยู่ด้วยกันหลายวิธี เช่น Inverted index, Vectorspace, N-Gram ซึ่งแต่ละวิธีก็มีข้อดีข้อเสียต่างกัน แต่วิธีที่เป็นที่นิยมมากสำหรับเสิร์ชเอนจินก็คือ Inverted index เว็บเพจใดๆจะสามารถสืบค้นได้จากเสิร์ชเอนจินก็ต่อเมื่อเว็บเพจนั้นผ่านการทำดัชนีมาแล้วเท่านั้น

2.3.3 ซอฟต์แวร์เสิร์ชเอนจิน (Search engine software)

เป็นโปรแกรมทำหน้าที่รับคำศัพท์ที่ผู้ใช้ต้องการค้นหาผ่านทางซีจีไอ (CGI, Common Gateway Interface) มาค้นหาในดัชนีค้นหาที่ถูกสร้างโดยอินเด็กเซอร์เพื่อ

ค้นหาเว็บเพจที่ตรงกับความต้องการของผู้ใช้เสิร์จเอนจินบางระบบจะจัดอันดับความเกี่ยวข้องของเว็บเพจด้วย



รูปที่ 2.1 การทำงานของส่วนประกอบแต่ละส่วนในระบบเสิร์จเอนจิน

2.4 Web robot (Web Crawler)

คือโปรแกรมซึ่งจะทำการดึงทุกๆหน้าใน World Wide Web หรือระบบเครือข่ายอินเทอร์เน็ตโดยการกระจายตัว Hyperlink ที่อ้างถึงจากหน้าแรกแล้วทำการดึงข้อมูลมาโดยที่การทำงานของเว็บโรบอท จะต้องขึ้นกับ robots.txt เป็นโปรโตคอล ซึ่งแต่ละเว็บไซต์จะมีเป็นของตัวเองเพื่อที่จะบอกว่าสามารถเข้าไปยังไฟล์ใดได้บ้างแต่โรบอทก็มีข้อเสีย อาทิเช่น

- โรบอททำให้เครือข่ายทำงานหนักมากขึ้น ซึ่งส่วนมากจะเกิดกับคนที่เริ่มเขียนโรบอท แต่ปัจจุบันมีข้อมูลเพียงพอที่จะป้องกันความผิดพลาดนี้แล้ว
- โรบอทถูกสร้างขึ้นโดยมนุษย์ ซึ่งถูกกระทำอย่างผิดพลาดในการตั้งองค์ประกอบ หรือไม่ได้พิจารณาการกระทำที่แฝงมาภายหลัง

เว็บโรบอทบางครั้งอาจจะถูกกล่าวถึงในชื่อ Web Wanderers, Web Crawlers หรือ Spiders

Web robot มีด้วยกันหลายชนิดดังนี้

- indexing
- HTML validation
- Link validation
- “ What ’ s new ” monitoring
- Mirroring

ซึ่งในที่นี้จะอธิบายเพียงอินเด็กซ์โรบอทซึ่งจะใช้ในเสิร์จเอนจิน

2.4.1 Indexing Robot

โดยทั่วไปโรบอทจะเริ่มทำงานจากรายชื่อของ URL ที่เก็บไว้เดิม โดยเฉพาะไฟล์ที่มีการเชื่อมโยงไปที่อื่นมากๆ อย่างเช่นเซิร์ฟเวอร์ใหม่ๆ และเว็บไซต์ที่ได้รับความนิยม บริการของ Indexing ส่วนมาก จะอนุญาตให้เลือก URL เองซึ่งจะเข้าไปในคิวและจะถูกเข้าถึงโดยโรบอท

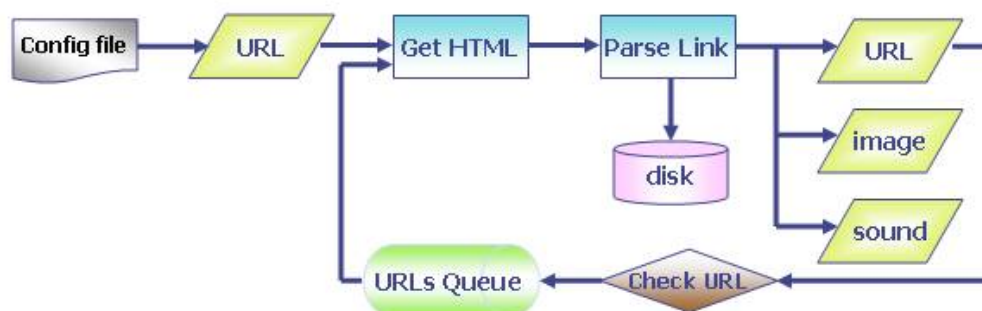
Indexing robot จะตัดสินใจว่าจะไรคืออินเด็กซ์โดยที่ถ้ารู้ข้อมูลเกี่ยวกับไฟล์นั้น Indexing robot อาจจะตัดสินใจที่จะตัดทิ้ง และทำการแทรกตัวเองเข้าไปในฐานข้อมูล ซึ่งการจะทำการไรก็ขึ้นอยู่กับโรบอทบางโรบอท จะเก็บ title เป็นอินเด็กซ์ หรือ 2-3 ย่อหน้าแรก หรือว่าอาจจะเก็บทุกๆคำ ซึ่งน้ำหนักของสิ่งที่จะเก็บก็ขึ้นอยู่กับโครงสร้างของ HTML บางส่วนจะเก็บ meta tag หรือ tag พิเศษอื่นๆ ที่ถูกซ่อนอยู่ ซึ่งโรบอทจะสามารถเข้าไปโหลดได้ก็ต่อเมื่อเข้าไปตรวจสอบที่ไฟล์ robot.txt ซึ่งเป็นไฟล์ที่จะบอกว่าสามารถเข้าถึงส่วนใดได้บ้างของไซต์นั้น ซึ่งแต่ละเว็บไซต์ก็จะสามารถกำหนดโปรโตคอล robot.txt ได้เฉพาะตัว

2.4.2 การทำงานของโรบอท

โรบอทเป็นโปรแกรมที่ต้องไปในอินเทอร์เน็ตเพื่อจุดประสงค์ใดจุดประสงค์หนึ่งเช่น เพื่อตรวจสอบว่ามีเว็บเพจใดบ้างที่ไม่สามารถอ่านได้ เป็นต้น แต่วัตถุประสงค์หลักของโรบอทที่เกี่ยวกับเสิร์จเอนจินก็คือ โรบอทจะทำหน้าที่รวบรวมข้อมูลจากเว็บต่างๆ จากอินเทอร์เน็ต แล้วบันทึกไฟล์เหล่านั้นลงในฮาร์ดดิสก์เพื่อนำมาใช้ในส่วนของอินเด็กซ์ต่อไป

ในปัจจุบันได้มีมาตรฐาน Standard Robot Exclusion ซึ่งเป็นมาตรฐานที่ใช้กำหนดการทำงานของโรบอท สำหรับเว็บไซต์แต่ละแห่งโดยที่เว็บไซต์ใดๆสามารถกำหนดได้ว่าจะยอมให้โรบอทตัวใดมาอ่านหรือไม่อนุญาตให้อ่านข้อมูลส่วนหนึ่งส่วน

การทำงานของโรบอทโดยทั่วไปจะมีลักษณะใกล้เคียงกันคือ เริ่มอ่านเว็บเพจ เริ่มต้นแล้วตรวจสอบดูว่าภายในเว็บเพจที่อ่านมาได้นั้นมีไฮเปอร์ลิงค์อะไรบ้าง เคยพบมาก่อนหรือไม่ ถ้ายังไม่เคยพบมาก่อน ก็จะมาวิเคราะห์ดูว่าสมควรที่จะอ่านเว็บเพจนั้นหรือไม่ โดยจะตรวจสอบโดยอาศัยกฎที่มีการตั้งไว้ เช่น จะอ่านเว็บไซต์ใดและไม่อ่านเว็บไซต์ใดบ้าง เมื่อเว็บเพจนั้นผ่านกฎที่มีการตั้งไว้แล้ว ตัวโรบอทก็จะตามไปอ่านเว็บเพจนั้นมา ทำซ้ำเช่นนี้จนกว่าลิงค์ที่ได้มาจะอ่านได้ครบ ดังรูปที่ 2.2



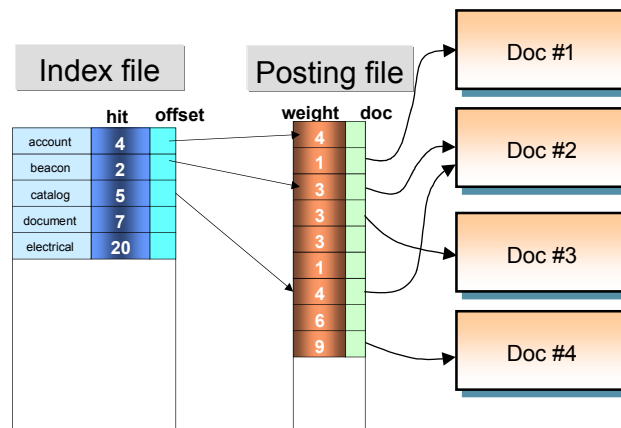
รูปที่ 2.2 การทำงานของโรบอท

2.5 อินเด็กเซอร์

การทำงานของโปรแกรมอินเด็กซอร์นั้นมีวิธีในการสร้างดัชนีค้นหาอยู่หลายวิธี เช่น แบบอินเวอร์ตอินเด็กซ์ (Inverted index), เวกเตอร์สเปซ โมเดล (vector space model) เป็นต้น แต่ในที่นี้จะใช้เทคนิคในการสร้างดัชนีค้นหาแบบอินเวอร์ตอินเด็กซ์ ตัวอย่างเช่น สมมุติว่ามีหนังสือหนาๆอยู่เล่มหนึ่งแล้วต้องการค้นหาคำว่า “network” ในหนังสือเล่มนั้น คงไม่มีใครที่จะเปิดหนังสือทีละหน้า อ่านตัวหนังสือทีละบรรทัด เพื่อที่จะดูว่ามีคำว่า “network” อยู่ที่หน้าไหน สิ่งที่จะทำก็คือเปิดหนังสือไปที่ท้ายเล่มเพื่อดูดัชนีคำศัพท์ของหนังสือเล่มนั้น ว่ามีคำว่า “network” อยู่ที่หน้าไหน รูปแบบของดัชนีค้นหาของอินเวอร์ตอินเด็กซ์ก็คล้ายๆกันกับดัชนีท้ายเล่มของหนังสือ อินเวอร์ตอินเด็กซ์ก็คือตารางของคำศัพท์ที่ถูกสร้างขึ้นมามาไว้สำหรับค้นหา โดยที่ภายในของอินเวอร์ตอินเด็กซ์นั้นจะจัดเก็บคำศัพท์และตำแหน่งที่อยู่ของคำศัพท์เหล่านั้นว่าอยู่ที่ใด เพราะฉะนั้นเมื่อมีการค้นหาเกิดขึ้นก็ไม่จำเป็นต้องไปค้นหาคำศัพท์จากเอกสารทีละเอกสาร แต่สามารถดูที่อินเวอร์ตอินเด็กซ์ได้เลยว่าคำศัพท์คำที่ค้นหาอยู่ที่ไหนบ้าง

โปรแกรมอินเด็กเซอร์นั้นจะมีหน้าที่สร้างตารางอินเวอร์คินเด็กซ์ ข้อมูลในอินเวอร์คินเด็กซ์ นั้นนอกจากจะเก็บตำแหน่งที่อยู่ของคำศัพท์แล้ว อินเด็กเซอร์ยังมีหน้าที่เก็บข้อมูลลักษณะรูปแบบของคำศัพท์แต่ละคำพบว่าปรากฏอยู่ใน HTML Tag อะไรเพื่อเป็นข้อมูลให้กับเสิร์จเอนจิน ซอฟต์แวร์ นำเอาไปใช้ในการจัดเรียงลำดับตอนค้นหาต่อไป

Inverted Index



รูปที่ 2.3 รูปแบบการจัดเก็บข้อมูลของโปรแกรมสร้างดัชนี

2.6 ซอฟต์แวร์เสิร์จเอนจิน

ซอฟต์แวร์เสิร์จเอนจินมีหน้าที่รับคำศัพท์ที่ต้องการค้นหาจากผู้ใช้ มาค้นหาในดัชนีค้นหา แล้วจัดเรียงลำดับผลลัพธ์ให้ได้ตรงกับความต้องการของผู้ใช้ให้มากที่สุด โดยอาศัยข้อมูลที่ได้เตรียมไว้ก่อนแล้วจากโปรแกรมอินเด็กเซอร์

การค้นหาแต่ละครั้งนั้น โปรแกรมค้นหาจะนำคำแต่ละคำในชุดวลีค้นหา (Query string) ที่ผู้ใช้ป้อนเข้ามามาเปรียบเทียบกับคำในตารางอินเวอร์คินเด็กซ์ เพื่อหาเอกสารที่มีคำศัพท์นั้นๆ ปรากฏ หลังจากที่ได้ชุดของเอกสารที่มีคำแต่ละคำมาแล้ว โปรแกรมค้นหาจะนำชุดของเอกสารแต่ละชุดมารวบรวมเข้าด้วยกัน แล้วจัดเรียงลำดับของเอกสารที่มีค่าน้ำหนักการค้นหาที่สูงไปน้อยที่สุด หลังจากนั้นโปรแกรมค้นหาจะส่งข้อมูลของเอกสารที่ได้มีการจัดเรียงลำดับเรียบร้อยแล้วไปยังผู้ใช้

เทคนิคที่ใช้ในการจัดอันดับก็คือ จะอาศัยจำนวนคำศัพท์และลักษณะรูปแบบของคำศัพท์นั้น ยกตัวอย่างเช่น หากมีเอกสารอยู่สองเอกสารคือ เอกสาร A และเอกสาร B เอกสาร A มีคำว่า network ปรากฏอยู่ทั้งหมด 4 แห่งในเอกสาร เอกสาร B มีคำว่า network ปรากฏอยู่ทั้งหมด 10 แห่ง เพราะฉะนั้นหากยูสเซอร์ค้นหาคำว่า network เสิร์จเอนจินก็จะแสดงผลลัพธ์โดยที่ลำดับของ

ในการทำงานเดียวกันลักษณะรูปแบบของคำศัพท์ก็มีผลในการจัดอันดับคือ หากเอกสาร A มีคำว่า network อยู่ที่ Title ของเอกสารแต่เอกสาร B มีคำว่า network อยู่ที่ท้ายเอกสาร เพราะฉะนั้นหากผู้ใช้ค้นหาคำว่า network ลำดับของผลลัพธ์ที่ได้เอกสาร A ก็ควรอยู่ก่อนเอกสาร B

ชนิดของ Tag	นัยความสำคัญ
URL	10
TITLE	100
ANCHOR	20
META	50
B , I , U , H1 , H2 , H3	20
Font	1

ตารางที่ 2.1 ตัวอย่างการกำหนดค่านัยสำคัญของแต่ละ TAG

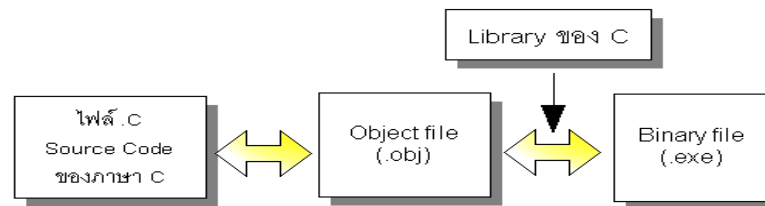
2.7 หลักการทำงานของภาษา Python

การใช้ภาษา Python ในการพัฒนา web application นี้เพราะ Python เป็นภาษาระดับสูง ภาษาหนึ่งที่เป็น object-oriented ที่มีความสามารถสูงถูกสร้างขึ้นในปี 1989 โดย Guido van Rossum ที่รวมเอาความสามารถแบบ dynamic ซึ่งทำให้เป็นที่น่าสนใจในการพัฒนา application ได้อย่างรวดเร็ว เช่นเดียวกับการใช้ในรูปแบบสคริปต์เพื่อเชื่อมต่อกับ component ที่มีอยู่แล้วเข้าด้วยกัน ทั้งนี้ยังเป็นภาษาที่ง่ายต่อการเรียนรู้ และยังสนับสนุนโมดูลและแพ็คเกจต่างๆซึ่งทำให้สามารถนำมาประกอบใช้ได้ใหม่ ซึ่งภาษานี้พัฒนาขึ้นมาโดยไม่ยึดติดกับแพลตฟอร์ม กล่าวคือสามารถรันภาษา Python ได้ทั้งบนระบบ UNIX, Linux, Windows NT/2000/XP/2003 หรือแม้แต่ระบบ FreeBSD อีกอย่างหนึ่งภาษาตัวนี้เป็นภาษาลักษณะ Open Source ที่แจกจ่ายให้ใช้งานฟรี

เมื่อทำการเขียนโค้ดขึ้นมาตามโครงสร้างของโปรแกรมภาษาใดก็ตาม และการจะให้โค้ดคำสั่งเหล่านั้นทำงานได้ก็จะต้องมีตัวแปลภาษามาจัดการแปลโค้ดคำสั่ง เพื่อให้ทำงานตามที่ต้องการ

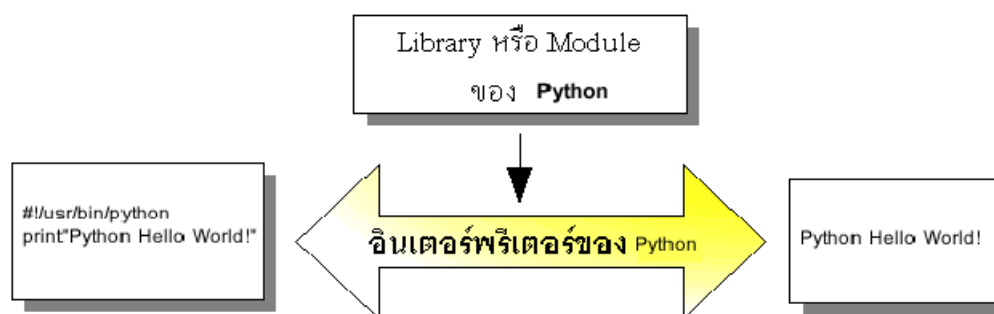
โดยลักษณะของตัวแปลภาษานั้นแบ่งได้ 2 ประเภทใหญ่ๆ คือ

2.7.1 คอมไพเลอร์ (Compiler) เป็นตัวแปลภาษาสำหรับภาษา C, C++, Pascal การทำงานก็คือจะตรวจสอบความผิดพลาดของโค้ดคำสั่งตั้งแต่ต้นจนจบก่อน หรือเรียกว่าการคอมไพล์ ถ้าไม่มีข้อผิดพลาดก็จะทำการแปลโค้ดคำสั่งให้เป็นไฟล์นามสกุล .obj (object file) จากนั้นก็ทำการแปลไฟล์ .obj ให้เป็นไบนารีไฟล์ .exe เพื่อทำงานต่อไป ดังตัวอย่างการทำงานของคอมไพเลอร์ภาษา C ดังรูป



รูปที่ 2.4 การทำงานของคอมไพเลอร์

2.7.2 อินเตอร์พรีเตอร์ (Interpreter) จะทำงานเป็นบรรทัดต่อบรรทัด คืออ่านโค้ดคำสั่งมาบรรทัดหนึ่งแล้วก็ทำงานให้ผลออกมาเลย ดังแสดงในรูป



รูปที่ 2.5 การทำงานของอินเตอร์พรีเตอร์

จากรูปตัวอย่างในกรณีที่มีการเรียกใช้ฟังก์ชันจากไลบรารี (Library) หรือโมดูล (Module) ของภาษา Python อินเตอร์พรีเตอร์ของภาษา Python ก็จะไปทำการเรียกฟังก์ชันเหล่านั้นให้ทำงานแล้วจึงแสดงผลการทำงานออกมา

ในส่วนประสิทธิภาพการทำงานนั้นตัวแปลภาษาแบบคอมไพเลอร์จะทำงานได้เร็วกว่าตัวแปลภาษาแบบอินเตอร์พรีเตอร์เพราะโค้ดคำสั่งถูกคอมไพล์และลิงก์โดยตัวแปลภาษาแบบคอมไพเลอร์ก่อนแล้วจึงรันไฟล์ .exe ของโค้ดคำสั่งนั้นซึ่งได้ผ่านการทำการเชื่อมโยงเรียบร้อยแล้ว

This document was created by the trial version of Print2PDF.

Once Print2PDF is registered, this message will disappear.

Purchase Print2PDF at <http://www.software602.com>

อีกสิ่งหนึ่งที่ใช้ในการพัฒนา คือ Regular Expression (หรือ REs) นั้นเป็นภาษาโปรแกรม มีชนิดพิเศษที่มีขนาดเล็กโดยถูกฝังตัวไว้ภายใน Python โดยการใช้งาน Regular Expression นี้ ต้องทำการระบุกฎเกณฑ์สำหรับกลุ่มของสตริงที่เป็นไปได้ที่ต้องการจะทำการตรวจจับ โดยที่กลุ่มสตริงเหล่านี้จะเป็นประโยชน์กับภาษาอังกฤษ เช่น e-mail address เป็นต้น และยังสามารที่จะหาความเหมือนของรูปแบบสตริงที่กำหนดว่าอยู่ในตำแหน่งใดหรือใช้ REs ในการแก้ไขสตริง หรือแยกเป็นส่วนๆได้หลายวิธี โดยนำ Regular Expression มาใช้ในส่วนของการ application ดังนี้

เมื่อต้องการที่จะดึงแท็ก Html ที่เกี่ยว link ของรูปภาพในเว็บไซต์ โดยในที่นี้เป็นการจะเป็นการดึงมาจาก <http://www.sanook.com> แท็กที่อยู่ในหน้านั้นเป็นดังนี้

```

```

ในตอนแรกต้อง import re ซึ่งเป็น Library ของภาษา Python ที่ไว้เรียกใช้งาน Regular Expression และเพื่อที่จะได้ส่วน URL ที่ต้องการโดยเมื่อจะทำงานต้องใช้ฟังก์ชัน match ค่าของ Regular Expression คือ `re.compile("src=('[\S+]*[a-zA-Z0-9_]+*(.jpg|.bmp)(/)*(')*)", re.IGNORECASE)` โดยที่ `"src=('[\S+]*[a-zA-Z0-9_]+*(.jpg|.bmp)(/)*(')*)"` เป็น pattern ที่ต้องการจะ match link ของรูปภาพ คือ src ในภาษา html โดยต้องสร้าง pattern นี้ ขึ้นมาเอง ต่อมาต้องค้นหา pattern ที่ต้องการในหน้าเว็บเพ็จนั้นโดยการใช้ฟังก์ชัน search ค้นหา `re.search(<pattern>, line)` โดยที่ `<pattern>` เป็น pattern ที่ต้องการค้นหา และ `line` เป็นส่วนที่จะทำการค้นหาถึงขั้นตอนนี้จะได้ผลลัพธ์ออกมาคือ `"/story_picture/s/03542_001.jpg"` เมื่อได้ผลลัพธ์ที่เป็น URL แบบนี้ออกมาแล้วจะยังไม่สามารถนำ URL นี้ไปเปิดในหน้าเว็บเพ็จได้ จึงต้องทำการรวม URL ในส่วนของ URL ที่เป็นหน้าหลักของเว็บนี้ และในส่วนผลลัพธ์ที่ได้จากการตัดคำมา โดยใช้ฟังก์ชันการทำงานของ Python คือ `urljoin(site,match_jpg_name)` โดยที่ `site` คือส่วนหน้าหลักของเว็บนั้น และ `match_jpg_name` คือส่วนผลลัพธ์ที่ได้มาจากการตัดคำได้ผลลัพธ์ที่เป็น URL ที่ใช้ในการลิงค์ไปหารูปภาพเป็น

http://www.sanook.com/story_picture/s/03542_001.jpg ซึ่งสามารถนำผลลัพธ์ที่ได้มาไปใช้ในส่วนอื่นๆต่อไป

ในส่วนของการดึงแท็ก Html ที่เกี่ยวข้องกับ link ย่อยๆ เช่น

```
<a href="http://www.kmitnb.ac.th" target="_blank"> ต้องใช้ฟังก์ชัน match ค่าของ Regular Expression คือ
```

```
re.compile("href=('[\S+]*[a-zA-Z0-9_]+*(.php|.com|.shtml|.html|.htm|.cgi|.th)
*(/)*(')*)", re.IGNORECASE) โดยที่
"href=('[\S+]*[a-zA-Z0-9_]+*(.php|.com|.shtml|.html|.htm|.cgi|.th)*(/)*(')*)"
```

เป็น pattern ที่ต้องการจะ match ลิงค์ย่อในหน้าเว็บเพจ คือ href ในภาษา Html โดยที่ต้องสร้าง pattern นี้ขึ้นมา ต่อมาต้องค้นหา pattern ที่ต้องการในหน้าเว็บเพจนั้น โดยการใช้ฟังก์ชัน search ค้นหา re.search(<pattern>,line) ได้ผลลัพธ์เป็น <http://www.kmitnb.ac.th>

2.8 ข้อมูลภาพดิจิทัล (Digital Image Data)

การทำการประมวลผลรูปภาพต้องศึกษาพื้นฐานของรูปภาพก่อนว่ามีการสร้างขึ้นได้อย่างไร ซึ่งในรูปภาพแต่ละรูปจะมีองค์ประกอบต่างๆมากมายแต่องค์ประกอบที่จะนำมาประมวลผลนั้นเรียกว่า “พิกเซล (Pixel)” พิกเซลนั้นจะมีลักษณะเป็นจุดสี่เหลี่ยม ซึ่งเป็นองค์ประกอบเล็กๆที่มีอยู่มากมายบนภาพการประมวลผลภาพต้องนำพิกเซลแต่ละรูปมาทำการประมวลผลโดยที่แต่ละพิกเซลจะประกอบด้วยรายละเอียดสีต่างๆมารวมกันเป็นพิกเซล ซึ่งเรียกกลุ่มของสีที่ใช้ว่า “Color Space” ซึ่งรูปแบบการบอกรายละเอียดของสีบนรูปภาพ โดยกลุ่มสีที่แสดงผลนั้นมีอยู่หลายแบบในที่นี้จะกล่าวถึงกลุ่มสีที่แสดงผลที่ใช้ในงานวิจัยนี้เท่านั้น

การเกิดภาพต่างๆบนจอคอมพิวเตอร์จะเกิดจากการเรียงตัวของจุดสี่เหลี่ยมสีต่างๆที่เรียกว่า “พิกเซล” ซึ่ง “พิกเซล” เป็นตัวกำหนดความคมชัดของภาพในจอคอมพิวเตอร์ ถ้ากำหนดให้ภาพแต่ละภาพมีจำนวน “พิกเซล” มากภาพก็就会有ความละเอียดมากขึ้นตามไปด้วย แต่จะทำให้ไฟล์มีขนาดใหญ่ขึ้นด้วยตามความละเอียดของภาพ

พิกเซล (Pixel มาจากคำว่า "Picture Element") เป็นหน่วยพื้นฐานของสีที่สามารถโปรแกรมได้บนจอแสดงผลหรือบนภาพ ขนาดทางกายภาพของพิกเซลจะขึ้นอยู่กับการกำหนดความละเอียดของจอภาพ ดังนั้นถ้าต้องการแสดงผลให้มีความละเอียดสูงสุดขนาดทางกายภาพของพิกเซลก็จะเท่ากับขนาดทางกายภาพของจุด (Dot Pitch หรือ Dot Size) บนจอภาพ แต่ถ้าตั้งความละเอียดไว้ต่ำกว่าค่าสูงสุด พิกเซลจะมีขนาดใหญ่กว่าขนาดทางกายภาพของจุดบนจอภาพ นั่นคือแต่ละพิกเซลจะใช้มากกว่าหนึ่งจุดในการแสดงผล

ในปัจจุบันนี้กลุ่มสีที่แสดงผลมีมากมายหลายแบบ เช่น กลุ่มสีที่แสดงผล RGB เป็นกลุ่มสีที่แสดงผลที่บอกแสดงลักษณะของสี 3 สี คือ สีแดง สีเขียว และสีน้ำเงิน, กลุ่มสีที่แสดงผล HSV เป็นกลุ่มสีที่แสดงผลที่บอกลักษณะของภาพโดยใช้ ระดับสี, ความเข้มของสี, ความสว่างของสี เป็นต้น แต่ในงานวิจัยนี้เลือกใช้กลุ่มสีที่แสดงผล RGB ในการแสดงรายละเอียดของภาพ

2.8.1 คุณสมบัติของกลุ่มสีที่แสดงผล

โดยทั่วไปแล้วภาพจะอยู่ในรูปแบบของกลุ่มสีที่แสดงผล RGB ซึ่งแทนสีทั้งหมดในกลุ่มสีที่แสดงผลด้วยเวกเตอร์ 3 มิติ จึงใช้กลุ่มสีที่แสดงผลนี้เป็นจุดเริ่มต้นของลักษณะเฉพาะของภาพได้โดยใช้ลักษณะเฉพาะของสี (Color Feature) จากนั้นค่อยทำการลดระดับของสีให้เหมาะสมต่อการทำฮิสโตแกรม คุณสมบัติดังกล่าวคือ

2.8.1.1 Uniformity

Uniformity เป็นคุณสมบัติที่ใช้ในการเปรียบเทียบความใกล้เคียงของเมตริกสี โดยกลุ่มสีที่แสดงผลจะมีคุณสมบัตินี้จะต้องคำนวณได้ง่าย อย่างเช่น การหาความใกล้เคียงของเมตริกสีโดยใช้ฟังก์ชันที่ไม่ขึ้นกับตำแหน่งในกลุ่มสีนั้น การทำการแปลงสีจะเป็นวิธีที่ทำให้กลุ่มสีมีคุณสมบัตินี้

2.8.1.2 Completeness

Completeness เป็นคุณสมบัติสำคัญที่กลุ่มสีที่แสดงผลจะต้องมีสีที่สามารถแยกแยะได้อย่างชัดเจน สามารถทำได้โดยการทำการลดระดับสี

2.8.1.3 Compactness

Compactness เป็นคุณสมบัติที่สีในกลุ่มสีที่แสดงผลจะต้องแตกต่างจากสีอื่นๆ ในกลุ่มสีที่แสดงผลเดียวกัน คุณสมบัตินี้จะเป็นตัวกำหนดมิติของภาพเพื่อให้แน่ใจว่า เมื่อผ่านการเปรียบเทียบจากกลุ่มสีที่แสดงผล RGB แล้วจะไม่มีสี 2 สีที่เหมือนกัน การลดระดับสีจะทำให้เกิดคุณสมบัตินี้

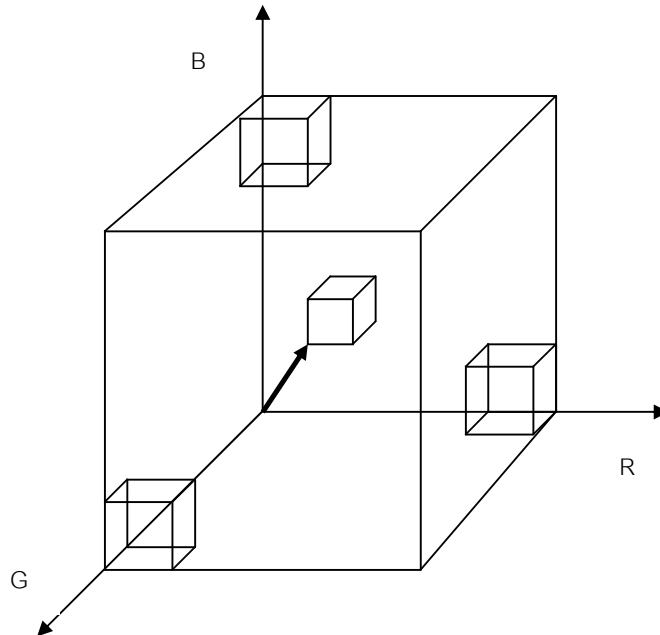
2.8.1.4 Naturalness

Naturalness เป็นคุณสมบัติของสีที่ผู้ใช้สามารถเข้าใจได้ง่าย ทำให้ผู้ใช้สามารถทำการดึงข้อมูลจากฐานข้อมูลโดยใช้สีเป็นดัชนีได้ ซึ่งสีดังกล่าวจะถูกแยกแยะออกเป็น 3 ส่วน คือ ความสว่าง (Brightness), ระดับสี (Hue), ความเข้มของสี (Saturation)

- ความสว่าง คือ คุณสมบัติที่มีผลกระทบโดยตรงต่อสีในด้านของแสง เช่น การเปลี่ยนแปลงระหว่างความสว่างถึงความมืด (Bright to Dim)
- ระดับสี คือ คุณสมบัติที่มีผลกระทบต่อสีที่จะทำให้สีเหมือนกับ สีแดง สีเหลือง สีเขียว สีนํ้าเงิน สีฟ้าสีม่วง ฯลฯ
- ความเข้มของสี คือ คุณสมบัติที่มีผลกระทบต่อสีหนึ่งของระดับสี โดยพิจารณาถึงปริมาณของสี (Color Light) โดยไม่สนใจถึงความสว่าง

2.8.2 กลุ่มสีแสดงผลแบบ RGB

ภาพสีที่อยู่ในรูปแบบของ RGB จะเป็นรูปแบบพื้นฐานของภาพที่ใช้แสดงบนจอคอมพิวเตอร์ โดยแต่ละ พิกเซลของภาพจะประกอบด้วยองค์ประกอบของสี 3 ค่า คือสีแดง, สีเขียว, สีนํ้าเงิน โดยแสดงองค์ประกอบสีละ n บิต ดังนั้นจะสามารถแสดงเฉดสีได้องค์ประกอบสีละ 2^n เฉดสีที่แตกต่างกัน รูปที่ 2.4 แสดงความสัมพันธ์ของสีแดง สีเขียว และสีนํ้าเงินใน 1 พิกเซล



รูปที่ 2.6 ความสัมพันธ์ของสีในกลุ่มสีแสดงผลแบบ RGB

ในระบบแกน R จะแสดงปริมาณของสีแดงที่มีในพิกเซลนั้น แกน G จะแสดงปริมาณของสีเขียวที่มีในพิกเซลนั้น และแกน B จะแสดงปริมาณของสีนํ้าเงินที่มีในพิกเซลนั้น เนื่องจากเก็บสีแยกเป็นค่าของแต่ละสีกันทำให้เมื่อมีการคำนวณต้องคำนวณแยกทีละค่าของสี

การผสมสีบนคอมพิวเตอร์นั้นอาศัยระดับสีหลัก 3 สีคือ สีแดง สีเขียว และ นํ้าเงิน หรือที่เรียกว่า “RGB Colors” (Red-Green-Blue) ความเหมือนจริงของสีคอมพิวเตอร์นั้น ขึ้นอยู่กับในหนึ่งจุดพิกเซลของการแสดงผลนั้นใช้ระดับของสี (Color depths) ที่มีค่าเท่าไร เช่นถ้าสี RGB มีระดับของสีเป็น 8 planes นั่นคือ ใช้ 8 บิตเก็บข้อมูลหนึ่งสี หมายความว่า เฉพาะระดับสีหลัก เช่น สีแดง สีเขียวก็จะเป็นค่าเป็นแดง อยู่ถึง $2^8 = 256$ ระดับ อธิบายง่ายๆก็คือ ถ้าจะผสมสี หนึ่งสี จากแดง-เขียว-นํ้าเงิน (RGB) จะใช้สีแดง ที่ส่วนจาก 0 ถึง 255 ส่วน ใช้เขียว ที่ส่วนจาก 0 ถึง 255 และเช่นเดียวกัน สีนํ้าเงินก็ส่วน จาก 0 ถึง 255 สีที่เกิดขึ้นก็จะเกิดจากการผสมของสีทั้งสาม ในอัตราส่วนต่างๆ กัน เช่น สีเหลืองธรรมชาติ เกิดจากการผสมสี แดง 255 ส่วน สีเขียว 255 ส่วน และสีนํ้าเงิน 0 ส่วน

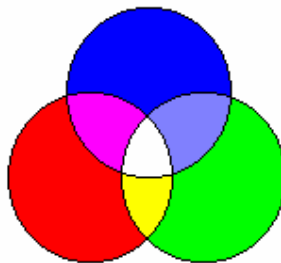
จากข้างต้น ระดับของสีแดง มีถึง 256 ระดับ สีเขียว 256 ระดับ สีนํ้าเงิน 256 ระดับ ดังนั้น RGB ทั้งหมดใช้ 24 บิต (8+8+8) ในการแสดงสี RGB ของหนึ่งจุดพิกเซลซึ่งสามารถแสดงสีได้มากถึง $256 \times 256 \times 256 = 16.7$ ล้านสี ตารางที่ 2.2 แสดงถึงจำนวนสีที่เป็นไปได้ ซึ่งขึ้นกับจำนวนบิตที่ใช้กำหนดระดับของสี และรูปที่ 2.8 แสดงการผสมสีของแม่สีในระบบ RGB

จำนวนบิตที่ใช้เก็บสีต่อหนึ่งจุด	Color Mode Name	จำนวนสีที่แสดงได้
1	Black and White	2
4	16-Color(EGA)	16
8	Pseudo Color	256
16	Hi-Color	65,536
24	True-Color	16,777,216

ตารางที่ 2.2 การกำหนดระดับสี

ค่าในแต่ละพิกเซลของสีของรูปภาพจะประกอบไปด้วยเวกเตอร์(Vector) ที่แสดงค่าของสีแดง สีเขียว และสีน้ำเงิน อย่างละ 8 บิต ดังนั้นรูปภาพ RGB 1 พิกเซล จะประกอบไปด้วยจำนวนบิตทั้งหมด 24 บิต ดังรูปที่ 2.7





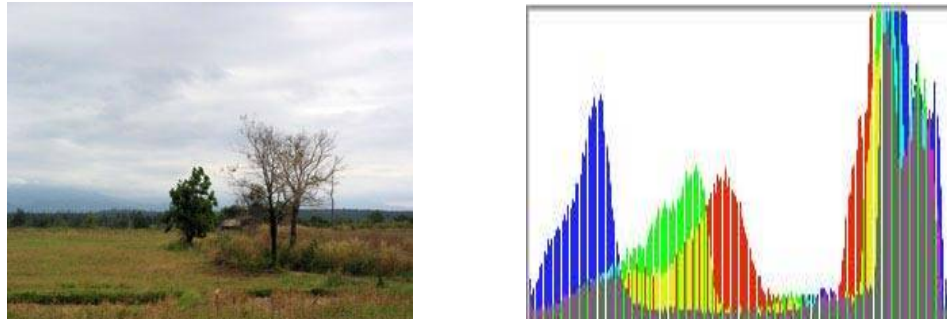
RGB

รูปที่ 2.8 การผสมสีในรูปแบบ RGB

2.9 ฮิสโตแกรม (Histogram)

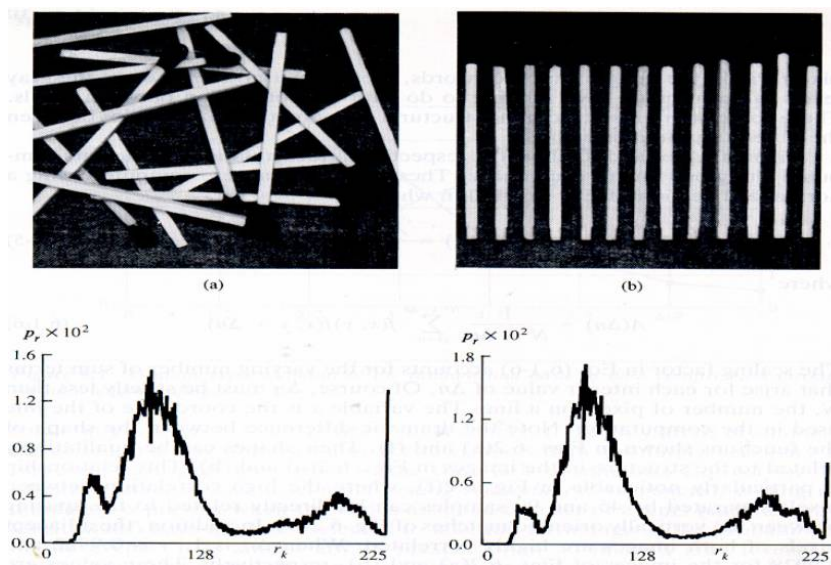
ฮิสโตแกรมของสีประกอบด้วยระดับต่างๆของสี ซึ่งระดับต่างๆนี้จะมีจำนวนของสีในแต่ละระดับที่ปรากฏในภาพ ฮิสโตแกรมของภาพสามารถอธิบายความมืด ความสว่าง ความคมชัดสูงหรือต่ำ ฮิสโตแกรมของสีจะมีความทนทานต่อการเปลี่ยนมุมมองของภาพเนื่องจากการเปลี่ยนมุมมองจะทำให้จำนวนพิกเซลรวมของสีไม่เกิดการเปลี่ยนแปลงมากนัก แต่ถ้ารูปร่างของภาพเมื่อมีการเปลี่ยนแปลงมุมมองแล้วจะเกิดการเปลี่ยนแปลงไปมาก ฮิสโตแกรมจะเป็นเสมือนฟังก์ชันในการกำหนดความน่าจะเป็นของสีที่ปรากฏในภาพ ซึ่งค่าของสีเดียวกันจะต้องอยู่ในระดับของสีเดียวกัน แต่ฮิสโตแกรมของสีอย่างเดียวไม่สามารถใช้ในการค้นหาภาพได้อย่างมีประสิทธิภาพ เนื่องจากว่าฮิสโตแกรมของสีไม่สามารถบอกได้ว่าระดับสีในแต่ละระดับอยู่ในตำแหน่งใดของภาพ อย่างไรก็ตาม ฮิสโตแกรมกลับได้รับความนิยมในการเป็นดัชนีของภาพ เพราะส่วนใหญ่แล้วพบว่าถ้าสีที่ปรากฏในรูปภาพมีความมืด เนื่องจากปัจจัยแวดล้อมต่างๆ ทำให้กลุ่มของสีที่ปรากฏในภาพเปลี่ยนตำแหน่งไป แต่ผลกระทบที่มีต่อ ฮิสโตแกรมของสีจะมีเพียงเล็กน้อยเท่านั้น ซึ่งทำให้การค้นหาภาพยังคงมีประสิทธิภาพที่ดีได้

ฮิสโตแกรมจะถูกแสดงด้วยกราฟโดยค่าที่แสดงถึงความเข้มพิกเซลในแต่ละระดับ ซึ่งจะเริ่มตั้งแต่ระดับ 0 ถึง 255 ระดับของพิกเซลจะถูกวางอยู่บนแกน x ส่วนค่าที่แสดงถึงความถี่ที่เกิดขึ้นซ้ำๆกันของสีในระดับนั้นจะถูกแสดงบนแกน y รูปที่ 2.9 จะแสดงภาพที่ทำฮิสโตแกรม



รูปที่ 2.9 ภาพที่ทำฮิสโตแกรม

สำหรับฮิสโตแกรม (Global) ของภาพ 2 ภาพที่มีรูปร่างกราฟคล้ายๆกัน ไม่ได้หมายความว่าภาพทั้ง 2 จะคล้ายกัน ทั้งนี้เพราะฮิสโตแกรมของภาพจะไม่เกี่ยวข้องกับตำแหน่งของจุดภาพ กล่าวคือ จุดภาพที่มีค่าระดับสีเหมือนกัน จากกราฟฮิสโตแกรมทั้ง 2 ไม่ได้ปรากฏอยู่ที่ตำแหน่งเดียวกันของภาพ ดังนั้นภาพทั้ง 2 จึงไม่เหมือนกัน ดังแสดงในรูปที่ 2.10



รูปที่ 2.10 ภาพที่มีกราฟฮิสโตแกรมเหมือนกันแต่ภาพต่างกัน

2.9.1 การ Quantization

ในงานวิจัยที่ได้จัดทำนี้ การสร้างฮิสโตแกรมของสีเพื่อเก็บสีที่ปรากฏอยู่ในภาพ โดยได้แบ่งค่า RGB ของรูปภาพถูกแบ่งออกเป็น 216 ระดับ ($R = 6$ ระดับ, $G = 6$ ระดับ, $B = 6$ ระดับ) เพื่อลดระดับความซับซ้อนของสีและเป็นการประหยัดเนื้อที่ในการจัดเก็บข้อมูลฮิสโตแกรมของแต่ละภาพ และทำให้ง่ายต่อการทำฮิสโตแกรม

ช่วงของสี	ค่าที่ใช้แทน
0 – 42	0
43 – 85	1
86 – 128	2
129 – 171	3
172 – 213	4
214 - 255	5

ตารางที่ 2.3 ระดับของสีที่ใช้ในการทำ Quantization

เริ่มด้วยต้องการหาว่าในพิกเซลที่สนใจนั้นประกอบไปด้วยค่าของสีอะไรบ้าง (RGB) เท่าไหร่ แล้วนำค่าที่ได้ไปเปรียบเทียบกับตารางว่าค่าของพิกเซลที่สนใจนั้นอยู่ในช่วงสีไหน แล้วใช้ค่า 0-5 แทนการแสดงด้วยค่าระดับสีที่แท้จริง ในสีแต่ละสีจะแบ่งค่าออกเป็นทั้งหมด 6 ช่วงสี ซึ่งเดิมจะถูกแบ่งไว้ทั้งหมด 255 สี เพราะฉะนั้นสามารถแสดงระดับสีจริงๆที่ผสมสีทั้ง 3 แล้วได้ทั้งหมด 216 ระดับ ตั้งแต่ค่า $R=0, G=0, B=0$ จนถึง $R=6, G=6, B=6$ เช่น ถ้า $R = 192, G = 64, B = 255$ ก็จะถูกรัน quantized เป็น $R = 4, G = 1, B = 5$

2.9.2 การ Normalization

เนื่องจากขนาดของรูปภาพจะมีผลทำให้จำนวนพิกเซลในแต่ละรูปภาพไม่เท่ากัน ส่งผลให้ค่าฮิสโตแกรมของรูปภาพ 2 รูปภาพที่มีขนาดไม่เท่ากันนั้นไม่สามารถจะนำมาเปรียบเทียบกันได้ ดังนั้นจึงจำเป็นต้องทำการ Normalization เพื่อให้สามารถนำมาเปรียบเทียบกันได้ โดยเมื่อหาค่าฮิสโตแกรมของรูปภาพได้แล้ว จะต้องนำความถี่ของแต่ละระดับสีมาหารด้วยจำนวนพิกเซลทั้งหมดที่มีในภาพ ผลที่ออกมาจะได้ค่าของความน่าจะเป็น (Probability) ของแต่ละระดับสีที่มีในรูปภาพ ซึ่งเมื่อรวมค่าความน่าจะเป็นของแต่ละระดับสีเข้าด้วยกันแล้วจะมีค่าเป็น 1 โดยสมการคือ

$$p(n) = \frac{f(n)}{P \times Q}$$

โดย

- $p(n)$ - ค่าความน่าจะเป็นของจำนวนพิกเซลที่มีระดับสีที่ n
 $f(n)$ - ค่าความถี่ของพิกเซลที่มีในระดับสีที่ n
 $P \times Q$ - จำนวนพิกเซลทั้งหมดที่มีในรูปภาพได้จากการนำจำนวน

พิกเซลตามความกว้างคูณกับจำนวนพิกเซลตามความยาว

2.9.3 ขั้นตอนการทำฮิสโตแกรม

หลังจากที่เลือกรูปภาพแล้วก็นำแต่ละภาพไปทำการ Quantization ซึ่งเป็นการลดระดับของสีลงเพื่อให้ง่ายต่อการคำนวณ โดยในงานวิจัยนี้จะแบ่งเป็น 6 ระดับของแต่ละสี คือ R 6 ระดับ G 6 ระดับ และ B 6 ระดับ รูปภาพที่ถูกทำ Quantization แล้วก็นำมาหาฮิสโตแกรมโดยนำแต่ละพิกเซลใน รูปภาพมาดูว่าอยู่ในระดับสีที่เท่าไร แล้วเก็บเป็นความถี่สะสมในแต่ละระดับสี เมื่อสแกนดูทุกพิกเซลในรูปภาพเสร็จแล้วก็จะนำความถี่สะสมของแต่ละระดับสีที่ทำการ Normalized โดยการนำความถี่สะสมของแต่ละระดับสีมาหารด้วยจำนวนพิกเซลทั้งหมด แล้วจะได้ผลลัพธ์ออกมาเป็น ความน่าจะเป็นของแต่ละระดับสี และนำค่าความน่าจะเป็นของแต่ละระดับสีคูณกับ 100 เพื่อให้ได้เป็นค่าเปอร์เซ็นต์ (%) หลังจากนั้นก็ปัดให้เป็นเลขจำนวนเต็มไปเก็บไว้ในอาร์เรย์ เพื่อใช้ในการค้นหาต่อไป รูปที่ 2.11 แสดงขั้นตอนการทำฮิสโตแกรม



รูปที่ 2.11 ขั้นตอนการทำฮิสโตแกรม